

Lab #1 TokTickIT Full-Stack Hello World Starter.

Name: วิศิษฐ์ สุวรรณเนา (Wisit Suwannao)

Student ID: 67070501042

GitHub username: Palapluem

Peer reviewer: นัธวัฒน์ ปริมสิริคุณาวุฒิ (Natthawat Primsirikunawut),

Student ID 67070501027, **GitHub username:** NOTAWOOD

Answer Part 1: Git Use with Engineering Workflow

TokTickIT was built as four required issues, each on its own feature/* branch off lab1-staging, reviewed by a real peer (NOTAWOOD), and merged with a merge commit. After all four were done, lab1-staging was released into main via one release PR #12, which merged the completed Lab 1 implementation into `main`. Documentation-only PRs #13, #14 and #15 were merged afterward.

Repository and Project URLs

What	URL
Repository	https://github.com/Palapluem/toktickit_cpe334
GitHub Project — TokTickIT Individual Sprints	https://github.com/users/Palapluem/projects/2

Issues

Issue	URL
#1 — Set up the TokTickIT project foundation	https://github.com/Palapluem/toktickit_cpe334/issues/1
#2 — Implement the API health check	https://github.com/Palapluem/toktickit_cpe334/issues/2
#3 — Create and seed IT request categories	https://github.com/Palapluem/toktickit_cpe334/issues/3
#4 — Display the IT request category list	https://github.com/Palapluem/toktickit_cpe334/issues/4

Pull Requests

PR	Branch flow	URL
#5	feature/1-project-foundation → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/5
#6	feature/2-health-check → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/6
#7	feature/3-category-seed → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/7
#8	lab1-staging → main	https://github.com/Palapluem/toktickit_cpe334/pull/8
#9	feature/4-category-list → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/9
#10 (Docs)	docs/lab1-test-evidence → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/10
#11 (Docs)	docs/ai-use-personalize → lab1-staging	https://github.com/Palapluem/toktickit_cpe334/pull/11
#12 (Release)	lab1-staging → main	https://github.com/Palapluem/toktickit_cpe334/pull/12
#13 (Docs)	docs/finalize-tests-evidence → main	https://github.com/Palapluem/toktickit_cpe334/pull/13
#14 (Docs)	docs/ai-use-personalize → main	https://github.com/Palapluem/toktickit_cpe334/pull/14
#15 (Docs)	docs/reviewer-update → main	https://github.com/Palapluem/toktickit_cpe334/pull/15

Note: PR #8 was a closed, unmerged release attempt. PRs #10 and #11 were documentation-only PRs merged into lab1-staging before PR #12. PRs #13, #14, and #15 were documentation-only PRs merged into main afterward.

GitHub Project board

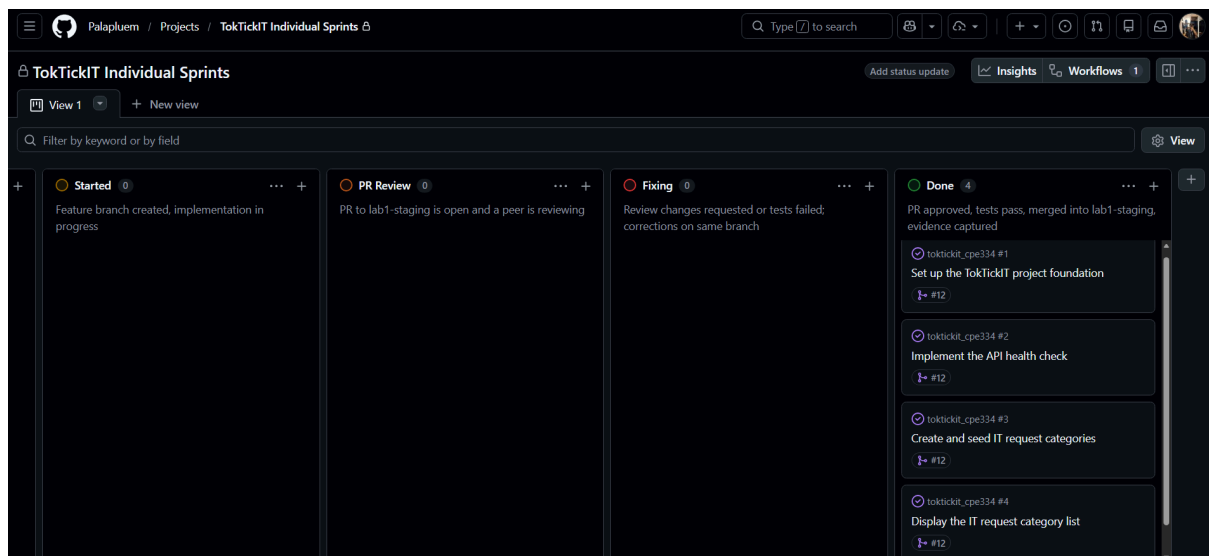


Figure 1.1 — GitHub Project "TokTickIT Individual Sprints" with all four issues in "Done" status.

Git workflow / commit graph

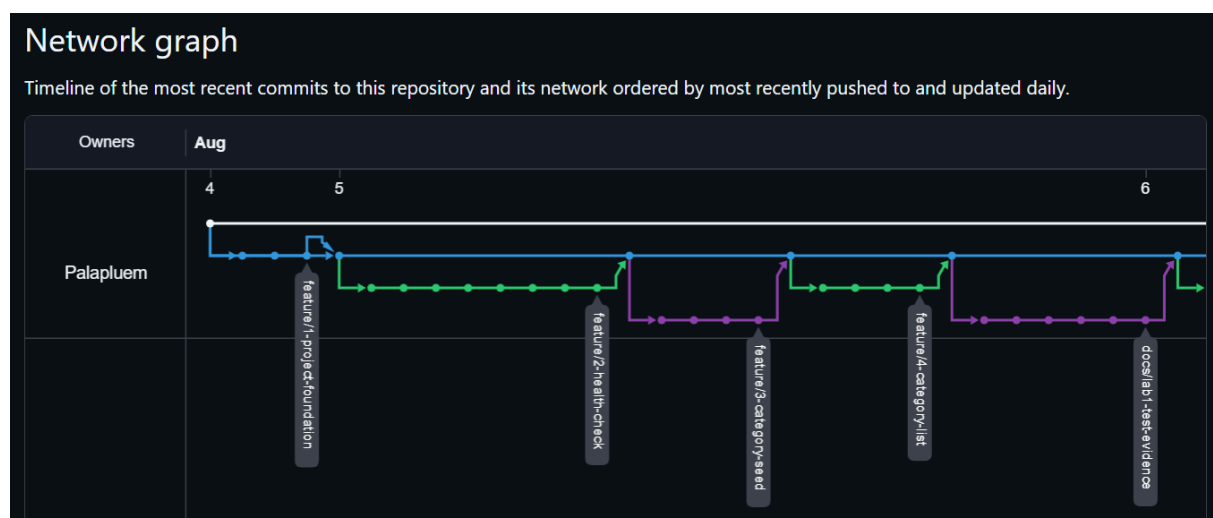


Figure 1.2a — Network graph: feature/1-project-foundation through feature/4-category-list branching from and merging into lab1-staging.

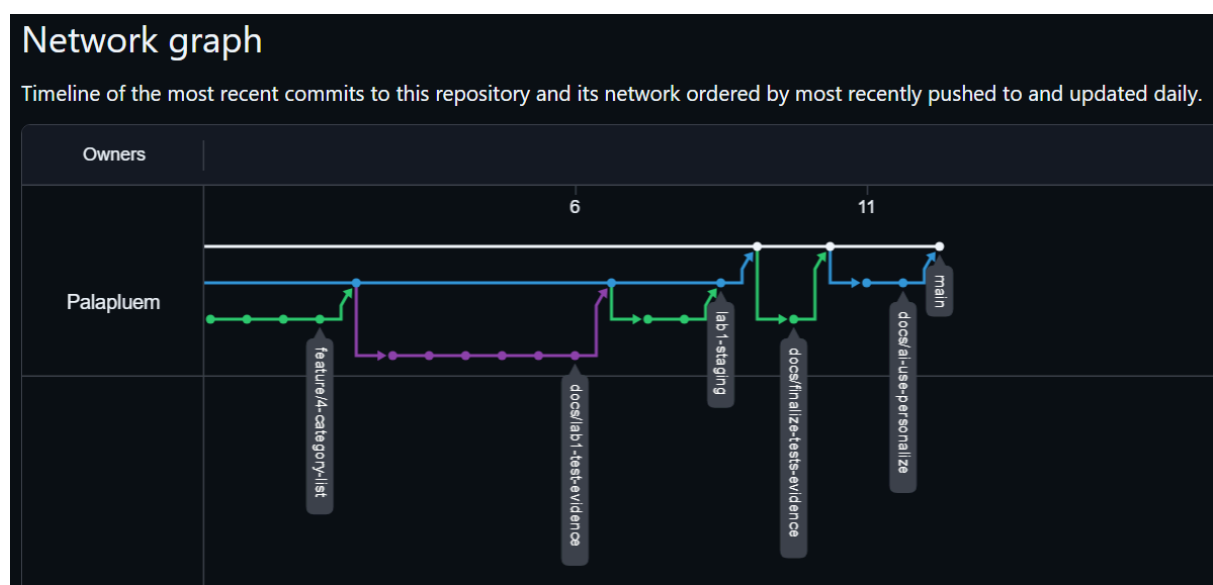


Figure 1.2b — Network graph continued: lab1-staging merging into main via the release pull request.

Note: This simplified network graph focuses on the four required feature branches and the lab1-staging release path. Documentation-only PRs are listed separately in the Pull Requests table.

Directory structure

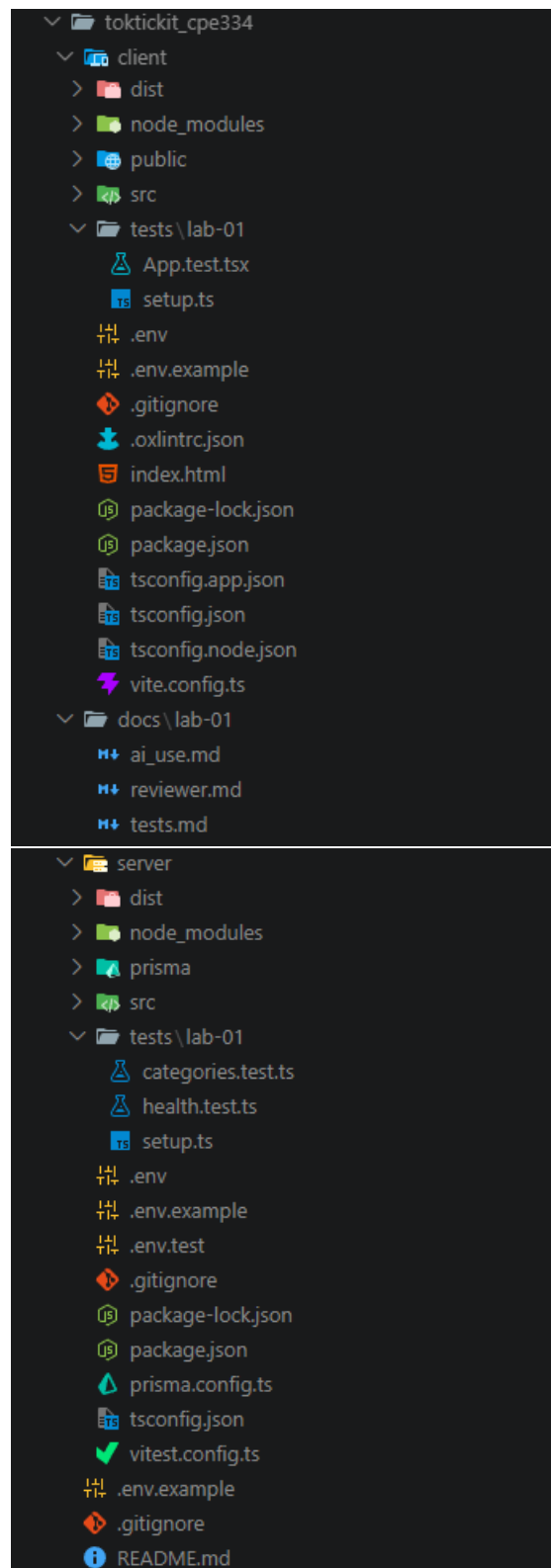


Figure 1.3 — Repository directory structure in the IDE, including docs/lab-01/ and the required test files under client/tests/lab-01/ and server/tests/lab-01/.

README.md and .gitignore

The screenshot shows a GitHub repository page for 'toktickit_cpe334' (Public). The repository has 8 branches, 0 tags, and 35 commits. A merge pull request #12 from Palaplum/lab1-staging is active. The file list shows a recent commit by Palaplum adding setup instructions for .env.example, .gitignore, and README.md. The README.md content is displayed, detailing the project's purpose, prerequisites, installation steps, and required environment variables. The right sidebar shows repository statistics and suggested workflows.

Repository Details:

- Repository: toktickit_cpe334 (Public)
- Branches: 8
- Tags: 0
- Commits: 35

File List (Recent Commits):

File	Commit Message	Time Ago
client	feat: display category list with loading and error states	20 hours ago
docs/lab-01	docs: record my real review of NOTAWOOD's PR	2 hours ago
server	feat: retrieve categories through Prisma	20 hours ago
.env.example	docs: add Lab 1 setup instructions	2 days ago
.gitignore	docs: add Lab 1 setup instructions	2 days ago
README.md	docs: add Lab 1 setup instructions	2 days ago

README.md Content:

TokTickIT — CPE334 Lab 1

Purpose

A tiny full-stack vertical slice — React UI → Express REST API → Prisma ORM → PostgreSQL — proving the required technology stack works together end-to-end. The app shows a `[Check System]` button that reports backend health and the four IT request categories stored in the database.

Prerequisites

- Node.js 20+ and npm
- PostgreSQL running locally, with a disposable local database available for development (`toktickit_dev`) and one for automated tests (`toktickit_test`)

Installation

```
cd server && npm install
cd ../client && npm install
```

Required Environment Variables

`.env.example` documents every variable; each package also has its own `.env.example` to copy from. Never commit the real `.env` files:

```
cp server/.env.example server/.env # DATABASE_URL, PORT
cp client/.env.example client/.env # VITE_API_BASE_URL
```

Repository Statistics:

- About: No description, website, or topics provided.
- Readme: Available
- Activity: Available
- Stars: 0
- Watching: 0
- Forks: 0

Packages: No packages published. [Publish your first package](#)

Contributors: 1 contributor: Palaplum Wisit Suwannao

Languages:

Language	Percentage
TypeScript	96.9%
HTML	2.9%
CSS	0.2%

Suggested workflows:

- Webpack: Build a NodeJS project with npm and webpack. By GitHub Actions. [Configure](#)
- Deno: Test your Deno project. By GitHub Actions. [Configure](#)
- SLSA Generic generator: Generate SLSA3 provenance for your existing release workflows. By Open Source Security Foundation (OpenSSF). [Configure](#)

[More workflows](#)

Figure 1.4 — Rendered README.md on the final main branch.

toktickit_cpe334 / .gitignore 

 Palapluem docs: add Lab 1 setup instructions 

Code Blame 29 lines (23 loc) · 327 Bytes

```
1  # Dependencies
2  node_modules/
3
4  # Environment variables – never commit real secrets
5  .env
6  .env.*
7  !.env.example
8
9  # Build output
10 dist/
11 build/
12 client/dist/
13 server/dist/
14
15 # Generated Prisma client
16 server/src/generated/
17
18 # Test coverage
19 coverage/
20
21 # Editor / OS noise
22 .vscode/
23 .idea/
24 .DS_Store
25 Thumbs.db
26
27 # Logs
28 npm-debug.log*
29 *.log
```

Figure 1.5 — .gitignore contents: excludes node_modules/, real .env files, build output, and the generated Prisma client.

PR review evidence

Peer review was real and bidirectional. On my four feature PRs, NOTAW00D reviewed every one; two (#6, #7) went through a genuine fixing round first—real defects were flagged (error swallowing, a missing error detail in the UI, and an inconsistent seed command), fixed on the same branch, and re-reviewed before approval. #5 and #9 were approved cleanly. In the other direction, I reviewed and approved NOTAW00D's PRs #5, #6, #7, #8, and #12; each was subsequently merged.

toktickit_cpe334 / docs / lab-01 / reviewer.md

Palapluem docs: update peer review evidencebb4e010 · 6 minutes agoHistory

PreviewCodeBlame38 lines (26 loc) · 5.39 KB

RawDownloadEditMore

Lab 1 Peer Review

Reviewer

- Name: นัทธวัฒน์ ปริณสิริคุณาวุฒิ (Natthawat Primsirikunawut)
- Student ID: 67070501027
- GitHub username: NOTAW00D

Reviews of My Pull Requests

PR	Reviewer	Decision	Review comment	My response	Evidence link
#5 — Set up the TokTickIT project foundation	NOTAW00D	Approved	"Igtm, your structure has been following the technical specs from SA docs (lab01-cheatsheet)." / "Ready to Merge!"	"Thanks for the review! Merged into lab1-staging with a merge commit."	#5
#6 — Implement the API health check (initial review)	NOTAW00D	Commented — changes requested	"Most part was great, still have an issues on error swallowing pls review it."	"Good catch — fixed in 990af85: the catch block now does <code>console.error('Health check failed:', error)</code> before setting the Offline state, instead of discarding the error. Added a test asserting the <code>console.error</code> call so it doesn't regress. UI message is unchanged. Ready for another look."	#6
#6 — Implement the API health check (round 2, inline comment on <code>client/src/App.tsx</code>)	NOTAW00D	Commented	"also display a part of error onto the react display comp"	"Done in f46e4fd — added an <code>errorDetail</code> state set from <code>error.message</code> (or 'Unknown error' as fallback), rendered as a muted 'Details: ...' line below the required 'Unable to connect to TokTickIT API' phrase. Added a test asserting the detail shows while the required phrase stays visible."	#6 (comment)
#6 — Implement the API health check (final review)	NOTAW00D	Approved after fixes	No additional summary comment; the final review state was APPROVED on commit <code>28d9c372</code> .	"The requested error logging and UI detail were implemented and re-reviewed. The PR was merged into lab1-staging as <code>deb2c2c2</code> ."	#6
#7 — Create and seed IT request categories (inline comment on <code>server/prisma.config.ts</code>)	NOTAW00D	Commented — clarification requested	"Does it create duplication of command like <code>npx</code> and <code>tsx</code> execution?"	"Good question. Not logic duplication — both entry points delegate to the single <code>prisma/seed.ts</code> ... There WAS an inconsistency though: this one used 'npx tsx' while <code>db:seed</code> used plain 'tsx'. Fixed in <code>de2138c</code> to drop the	#7 (comment)

#7 — follow-up	NOTAW00D	Commented	"If you have test the migration script to real db, it's fine for me 🙌."	"Yes — tested against real PostgreSQL, not mocked... SELECT COUNT(*), COUNT(DISTINCT name) FROM "Category" returned 4, 4 — no duplicates, correct id order."	#7 (comment)
#7 — Create and seed IT request categories (final review)	NOTAW00D	Approved after clarification	No additional summary comment; the final review state was <code>APPROVED</code> on commit <code>ed36dfde</code> .	"The command inconsistency and database-seeding clarification were addressed and re-reviewed. The PR was merged into <code>lab1-staging</code> as <code>e35fbf5e</code> ."	#7
#9 — Display the IT request category list	NOTAW00D	Approved	"These things look great."	(no changes requested — clean approve, merged as-is)	#9

My Reviews of Partner Pull Requests

Partner PR	My decision	My review comment	Partner response	Evidence link
NOTAW00D/TokTickIT #5 — Project foundation: React+Vite client, Express+Prisma server, Postgres, tests	Approved	"Well done Natthawat!, That's look good so far."	Merged, no further reply needed	NOTAW00D/TokTickIT#5
NOTAW00D/TokTickIT #6 — Add health check endpoint	Approved	"You are doing it well so far!"	Merged after review comments were addressed	NOTAW00D/TokTickIT#6
NOTAW00D/TokTickIT #7 — Add Category model, migration, and seeding	Approved	"Good job for this task!!!"	Merged after review comments were addressed	NOTAW00D/TokTickIT#7
NOTAW00D/TokTickIT #8 — Add category list endpoint and UI	Approved	"Good job, Natthawat! It's nice to see your final product in this repo for now!"	Merged after review comments were addressed	NOTAW00D/TokTickIT#8
NOTAW00D/TokTickIT #12 — Lab 1 Release (<code>lab1-staging</code> → <code>main</code>)	Approved	"Well done, Natthawat. Thanks for our good collaboration in this work!"	Merged into <code>main</code> after the release review	NOTAW00D/TokTickIT#12

Review Outcome

All four feature PRs (#5, #6, #7, #9) received real reviews from NOTAW00D and were merged into `lab1-staging` via merge commits. Two PRs (#6, #7) went through a real fixing cycle first — genuine defects or clarifications (error swallowing, a missing error detail in the UI, and an inconsistent seed command) were addressed on the same branches and re-reviewed before approval. PRs #5 and #9 were approved cleanly on the first pass. In the other direction, I reviewed and approved NOTAW00D's partner PRs #5, #6, #7, #8, and #12; each was subsequently merged.

Figure 1.6 — Rendered docs/lab-01/reviewer.md, showing reviewer identity and the full PR review record for both directions.

Full rendered content also available at

https://github.com/Palapluem/toktickit_cpe334/blob/main/docs/lab-01/reviewer.md

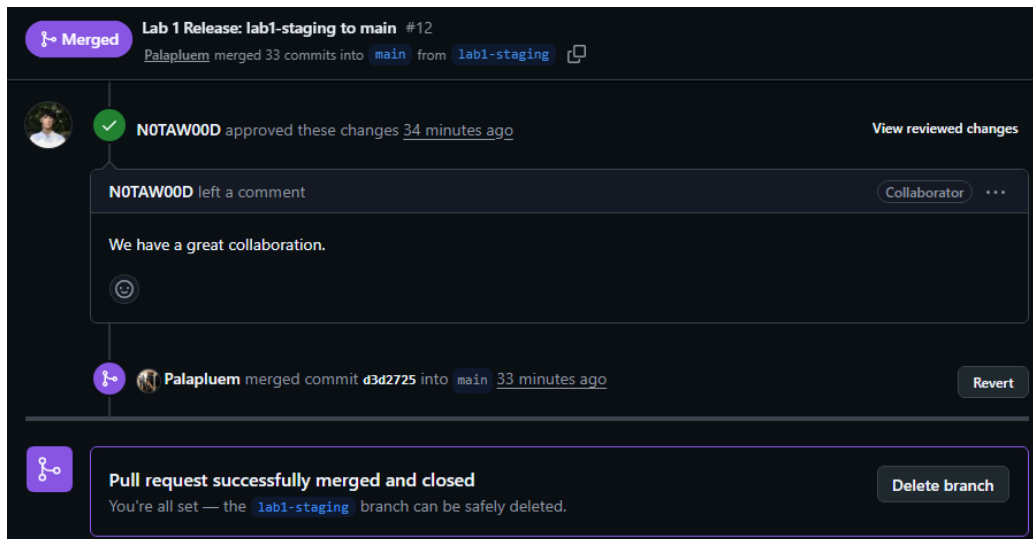


Figure 1.7 — NOTAWOOD's approval and merge confirmation on the release PR (#12, *lab1-staging* → *main*): "We have a great collaboration."

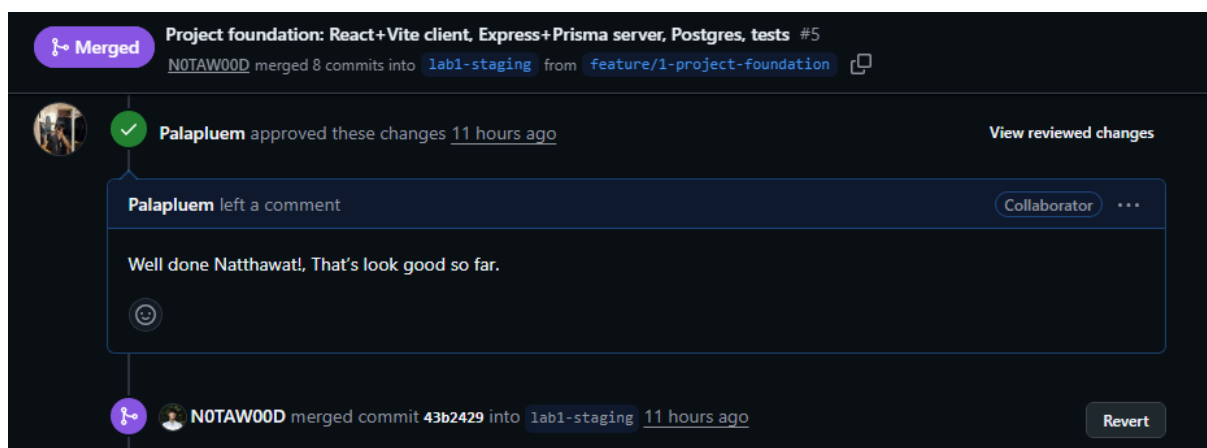


Figure 1.8 — My approval on NOTAWOOD's project-foundation PR (their repository, NOTAWOOD/TokTickIT#5): "Well done Natthawat!, That's look good so far."

What review comment did my partner give me, and how did I respond?

Example (PR #6): NOTAWOOD — "Most part was great, still have an issues on error swallowing pls review it." My response — "Good catch — fixed in 990af85: the catch block now does `console.error('Health check failed:', error)` before setting the offline state, instead of discarding the error." Full exchange across all four PRs recorded in `docs/lab-01/reviewer.md`.

What review comment did I give my partner, and how did they respond?

PR NOTAWOOD/TokTickIT#5 — "Well done Natthawat!, That's look good so far." → Approved, merged, no further reply needed.

Answer Part 2: Tests

All required tests pass. The application code was tested at commit d3d2725, the release commit from PR #12. Documentation-only PRs #13, #14, and #15 later advanced main to bf99c83 without changing the tested application code. The test suite covers 8/8 tests across two packages: API-01, API-02, UI-01, UI-02, and UI-03.

```
● PS D:\cpe334-software-engineer-2026> cd D:\cpe334-software-engineer-2026\labs\toktickit_cpe334
● PS D:\cpe334-software-engineer-2026\labs\toktickit_cpe334> cd server
● PS D:\cpe334-software-engineer-2026\labs\toktickit_cpe334\server> npm test

> server@0.0.0 test
> vitest run

RUN v4.1.10 D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/server

stdout | tests/lab-01/health.test.ts
◇ injected env (2) from .env.test // tip:  suppress logs { quiet: true }

stdout | tests/lab-01/categories.test.ts
◇ injected env (2) from .env.test // tip:  encrypted .env [www.dotenvx.com]

✓ tests/lab-01/health.test.ts (1 test) 27ms
✓ tests/lab-01/categories.test.ts (1 test) 331ms
  ✓ returns the four seeded categories from PostgreSQL via Prisma, in ascending id order 328ms

Test Files  2 passed (2)
Tests       2 passed (2)
Start at    16:55:29
Duration    1.44s (transform 237ms, setup 94ms, import 1.52s, tests 358ms, environment 0ms)
```

Figure 2.1 — server test output: 2/2 passed (API-01 health check, API-02 categories).

```

● PS D:\cpe334-software-engineer-2026\labs\toktickit_cpe334\server> cd ../client
● PS D:\cpe334-software-engineer-2026\labs\toktickit_cpe334\client> npm test

> client@0.0.0 test
> vitest run

RUN v4.1.10 D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client

stderr | tests/lab-01/App.test.tsx > UI-03: health check failure > shows a useful error message when the backend is unavailable
Health check failed: Error: network error
    at D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/tests/lab-01/App.test.tsx:24:33
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:302:11
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:1903:26
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2326:20
    at new Promise (<anonymous>)
    at runWithCancel (file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2323:10)
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2305:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2272:10)
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2955:64

stderr | tests/lab-01/App.test.tsx > UI-03: health check failure > shows a fragment of the real error alongside the required offline message
Health check failed: Error: Failed to fetch
    at D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/tests/lab-01/App.test.tsx:59:33
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:302:11
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:1903:26
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2326:20
    at new Promise (<anonymous>)
    at runWithCancel (file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2323:10)
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2305:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2272:10)
    at file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/runner/dist/chunk-artifact.js:2955:64

stderr | tests/lab-01/App.test.tsx > UI-03: health check failure > shows the offline error state when categories fail even though health succeeded
Health check failed: Error: categories down
    at D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/tests/lab-01/App.test.tsx:82:29
    at Mock (file:///D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/node_modules/@vitest/spy/dist/index.js:332:34)
    at fetchCategories (D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/src/api.ts:24:26)
    at handleCheckSystem (D:/cpe334-software-engineer-2026/labs/toktickit_cpe334/client/src/App.tsx:16:39)

✓ tests/lab-01/App.test.tsx (6 tests) 469ms
  ✓ UI-01: heading (1)
  ✓ renders the TokTickIT IT Service Desk heading 140ms
  ✓ UI-03: health check failure (4)
    ✓ shows a useful error message when the backend is unavailable 97ms
    ✓ logs the real error to the console for diagnostics instead of swallowing it 44ms

```

Figure 2.2a — client test output (detail view): UI-01 heading test and all four UI-03 error-state sub-tests.

```

  ✓ shows a fragment of the real error alongside the required offline message 62ms
  ✓ shows the offline error state when categories fail even though health succeeded 50ms
✓ UI-02: category list (1)
  ✓ shows a loading state, then the category list, after successful mocked API responses 74ms

Test Files  1 passed (1)
Tests       6 passed (6)
Start at    16:56:15
Duration    5.25s (transform 70ms, setup 556ms, import 221ms, tests 469ms, environment 3.58s)

```

Figure 2.2b — client test output (summary): 6/6 passed, including UI-02 category-list loading behavior.

Rendered docs/lab-01/tests.md (pasted as text below — copied from https://github.com/Palapluem/toktickit_cpe334/blob/main/docs/lab-01/tests.md):

None

Lab 1 Test Evidence

Environment

- Branch: `main` (final released state – release PR #12 merged `lab1-staging` into `main`)
- Commit: `d3d2725`
- Node version: v25.0.0
- npm version: 11.6.2
- PostgreSQL environment: two dedicated local databases on the same PostgreSQL 18 instance – `toktickit\_dev` for manual/dev use, `toktickit\_test` for automated Supertest runs (loaded via `server/.env.test`)
- Date executed: 2026-08-05 (re-confirmed against `main` on 2026-08-06
- code is byte-identical to the `lab1-staging` commit these results were originally captured from)

Test Catalog

ID	Test file	Tool	Description	Requirement	Result
API-01	`server/tests/lab-01/health.test.ts`	Supertest	`GET /api/health` returns 200 and `{ status: "ok", service: "TokTickIT API"}`	Issue 2	PASS
API-02	`server/tests/lab-01/categories.test.ts`	Supertest	`GET /api/categories` returns the four seeded categories via the real Prisma/PostgreSQL path, ascending `id` order	Issue 4	PASS
UI-01	`client/tests/lab-01/App.test.tsx`	Vitest	"TokTickIT IT Service Desk" heading renders	Issue 2	PASS
UI-02	`client/tests/lab-01/App.test.tsx`	Vitest	Clicking Check System shows a loading state, then the category list, after successful mocked API responses	Issue 4	PASS
UI-03	`client/tests/lab-01/App.test.tsx`	Vitest	API failure shows "Unable to connect to TokTickIT API" – covers a basic health failure, console diagnostic logging (not swallowed), an error-detail fragment shown in the UI, and categories failing even though health succeeded	Issue 2 / Issue 4	PASS

Commands and Output

```
```text
```

```
$ cd server && npm test
```

```
 RUN v4.1.10 .../server
```

```
 Test Files 2 passed (2)
```

```
 Tests 2 passed (2)
```

```
$ cd server && npm run build
```

```
> tsc -p tsconfig.json
```

```
(no errors)
```

```
$ cd client && npm test
```

```
 RUN v4.1.10 .../client
```

```
 Test Files 1 passed (1)
```

```
 Tests 6 passed (6)
```

```
$ cd client && npm run build
```

```
> tsc -b && vite build
```

```
✓ built in 339ms
```

```
```
```

Real (non-mocked) end-to-end confirmation, run against the seeded `toktickit\_dev` database with the server actually listening:

```
```text
```

```
$ curl http://localhost:3001/api/health
```

```
{ "status": "ok", "service": "TokTickIT API" }
```

```
$ curl http://localhost:3001/api/categories
```

```
[{ "id": 1, "name": "Account and
```

```
Access" }, { "id": 2, "name": "Hardware" }, { "id": 3, "name": "Software" }, { "id": 4, "name": "Network" }]
```

```
```
```

Idempotent seed, run twice back-to-back against `toktickit\_dev` (identical `id`/`createdAt` on both runs), cross-checked independently via `psql` (not through Prisma):

```
```text
```

```
$ psql -c "SELECT COUNT(*), COUNT(DISTINCT name) FROM \"Category\";"
```

```
count | count
```

```
-----+-----
```

## ## Manual Demo Checks

- Initial state: verified – `TokTickIT IT Service Desk` heading and `[Check System]` button render on `npm run dev`.
- Loading state: verified via UI-02 (deterministic assertion using a manually-gated promise) and via the real dev server (button shows "Checking..." while the request is in flight).
- Success state: verified end-to-end against the real server + seeded database (see curl output above) – `System Status: Online` plus all four categories in the required order.
- Failure state: verified via UI-03 (mocked network rejection) and manually during Issue 2 by stopping the real server and confirming the browser-equivalent fetch fails with `Unable to connect to TokTickIT API`.
- Browser Network-tab screenshot: captured – `System Status: Online` alongside DevTools Network tab showing `/api/health` and `/api/categories` both returning 200 (Part 4 App Demo evidence, see submission PDF).

### Answer Part 3: AI Use and Reflection

toktickit\_cpe334 / docs / lab-01 / ai\_use.md

2cc71a9 - 1 minute ago

History

Palaplum docs: refine AI prompts in professional English

35 lines (24 loc) · 5.76 KB

PreviewCodeBlameRawDownloadShare

# AI Use and Reflection - Lab 1

## Which agent and model I used

I used **Claude Code** in a VS Code-based IDE with **Claude Sonnet 5** at the default thinking level. The lab sheet says the course would most likely use Antigravity, but it also allows a VS Code-based IDE with an integrated AI coding assistant. I used Claude Code to read the requirements, plan the work, implement and test the application, explain review feedback, and verify Git/GitHub workflow decisions.

I did not treat the agent's first answer as something to follow blindly. I checked the requirements, repository state, generated files, test results, review feedback, and branch history before accepting changes.

## Tools and Models

- IDE/agent: Claude Code (VS Code extension)
- LLM/model: Claude Sonnet 5
- Thinking level or mode: default
- How AI was used: requirements analysis, implementation planning, full-stack coding, test design, debugging, peer-review discussion, and Git/GitHub workflow guidance

## Selected Key Prompts

Prompt Name	Prompt Text	My Reflection
Establish the Lab 1 execution contract	I am starting CPE334 Software Engineering Lab 1. Follow the complete instructions in <code>TokTickIT_Lab1_Master_Prompt.md</code> in detail. Treat this file as planning-only and do not commit it to either repository.	I set this boundary before implementation began because I wanted the requirements and repository rules kept separate from the graded source code.
Validate the feature-branch workflow	I currently see <code>main</code> , <code>lab1-staging</code> , and feature branches for Issues 1-4. Compare this state with the Lab 1 dependency order before creating or deleting anything. Should feature branches be created only when their predecessor is ready, or all at once? Do not copy another repository's branch layout without checking the rule.	I almost treated my friend's branch list as a template. This made me understand that branch timing matters because each feature must start from the correct staging state.
Confirm PR ownership and review gates	Please clarify the peer-review workflow for the Issue pull requests. Should my peer open PR #5 and the remaining PRs, or should I open each feature PR and ask my peer to review it? State the approval and merge gate that must be satisfied before the next dependent Issue starts.	I was confused about who creates a PR and who reviews it. Asking this clarified the workflow I followed for the rest of the lab.
Verify repository structure against the lab sheet	My peer says that the <code>docs/</code> directory is missing from <code>main</code> . Compare the repository with the lab sheet and the required graded-repository structure. Identify what is actually required, and exclude lecture-only materials from the graded repository.	I learned to compare the repository with the actual requirement instead of reacting to a single comment or copying another project's file layout.
Complete the same-branch review cycle	A reviewer has identified changes on the current feature branch. Confirm whether I must fix and verify those changes on the same branch before requesting another review, and explain what evidence should be preserved.	This showed me that review is a development gate, not just a comment to answer. The fix must remain connected to the PR being reviewed.
Investigate duplicate pull requests before merging	There are two pull requests in the repository. Inspect their base branches, states, and scope. Determine whether either one is an early release PR and which action is valid under the Lab 1 dependency order. Do not merge anything prematurely.	I noticed that the GitHub interface could make an early release look ready. Checking the branch targets first prevented an out-of-order merge.
Evaluate the late starter scaffold	The TA has provided a <code>Lab1_Starter_Scaffold</code> after several Issues have already been implemented and peer-reviewed. Compare it with the lab sheet and the current repository. Separate mandatory behavior from naming or formatting differences, and explain whether any change would invalidate reviewed Issues or require re-review.	I was worried that the late scaffold meant I had to rewrite completed work. Comparing requirements helped me avoid unnecessary changes and re-review.
Analyze the seed-command review question	The reviewer asked whether <code>npm run prisma:seed</code> creates duplicate command execution. Analyze the Prisma configuration, package scripts, and actual seed path. Determine whether a real defect exists, then draft a concise, evidence-based response. Change the code only if the analysis finds an actual problem.	I did not want to reply with a vague assurance without understanding the concern. The explanation connected the review comment to the actual command flow and migration behavior.

## Reflection on Improving My Prompts

My prompts became more effective when I named the artifact I was looking at, the Lab 1 rule involved, the uncertainty I needed to resolve, and the decision I wanted from the agent. At the beginning, I sometimes sent very short questions because the surrounding context was already shared. Later, I learned that a strong engineering prompt should make that context explicit instead of relying on the agent to guess.

The most important improvement was not making every prompt longer. It was making the expected outcome and constraints precise. For example, a question about two pull requests became an instruction to inspect their base branches and merge order, while a question about a reviewer comment became a request for an evidence-based analysis before any code change.

I still made the final decisions about branch order, pull-request state, review fixes, and whether the late scaffold actually required changes. The agent executed many steps, but the responsibility for understanding and approving the result remained mine.



*Figure 3.1 — Rendered docs/lab-01/ai\_use.md, showing the AI agent and model used, selected Lab 1 prompts, and my reflections on improving them.*

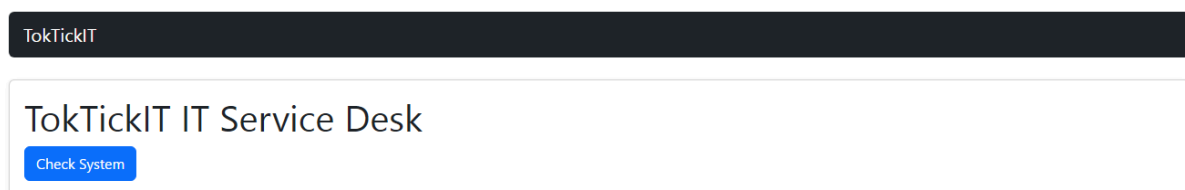
Full rendered content also available at

[https://github.com/Palapluem/toktickit\\_cpe334/blob/main/docs/lab-01/ai\\_use.md](https://github.com/Palapluem/toktickit_cpe334/blob/main/docs/lab-01/ai_use.md)

## Answer Part 4: App Demo

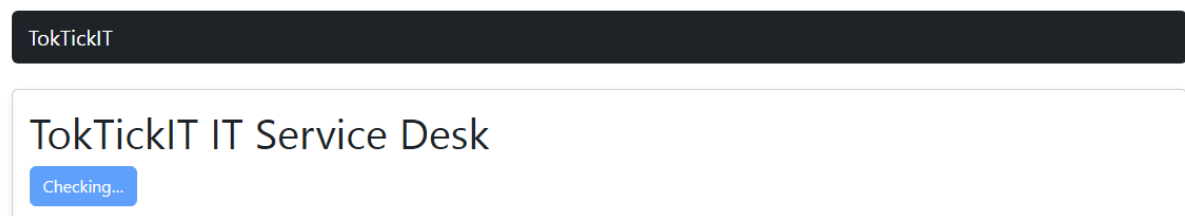
Demonstrated locally with the real backend and frontend running (npm run dev in both server/ and client/), against the seeded toktickit\_dev database.

### Initial state



*Figure 4.1 — Initial state: "TokTickIT IT Service Desk" heading with the [Check System] button, nothing else.*

### Loading state



*Figure 4.2 — Loading state ("Checking...") immediately after clicking [Check System], captured using DevTools network throttling to make the transient state visible.*

## Success case, after clicking [Check System]

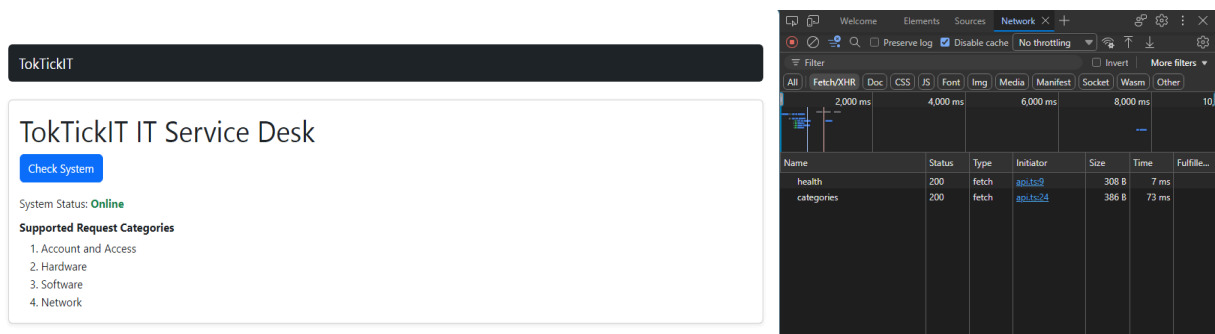


Figure 4.3 — Success state: System Status: Online with all four categories in the required order, alongside the DevTools Network tab confirming /api/health and /api/categories both returned HTTP 200.

## Failure case, after clicking [Check System]

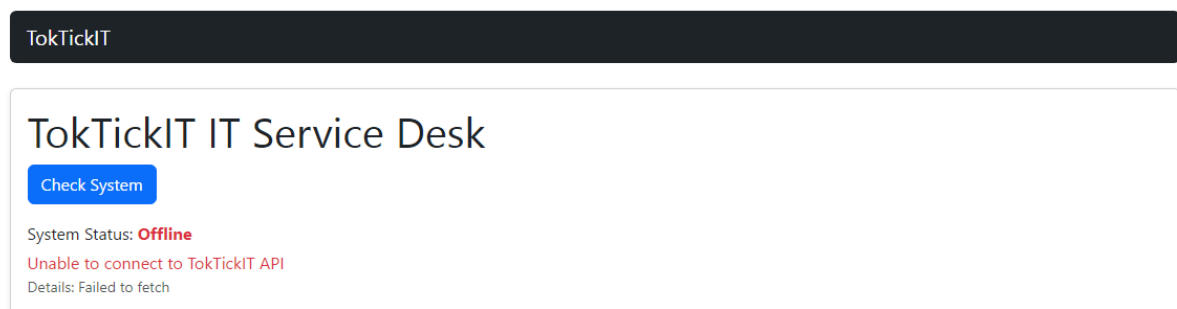


Figure 4.4 — Failure state: System Status: Offline with the required phrase "Unable to connect to TokTickIT API" plus the real error detail, captured by stopping the backend server and clicking [Check System] again. Service was restored immediately after capture.